СИМУЛЯЦИЯ ИНЖЕКЦИОННОЙ ЛИНИИ ПУЧКА ЗАРЯЖЕННЫХ ЧАСТИЦ

Бэлэнеску Богдан Викторович
Шакиров Артем Андреевич
Жарков Кирилл Алексеевич

ЦЕЛЬ

Смоделировать согласующую инжекционную линию ускорителя и подобрать параметры магнитной системы, обеспечивающие совпадение Twiss-параметров пучка с параметрами кольца коллайдера.

ЗАДАЧИ

- реализовать матричную модель движения частиц
- подобрать градиенты квадрупольных магнитов
- смоделировать траекторию центроида пучка
- визуализировать динамику пучка

ПАРАМЕТРЫ ИЗ ТЗ

ПАРАМЕТРЫ ПУЧКА

ВХОД:

$\beta x = 5$ м

$\alpha x = -0.5$

$\beta y = 2.5$ м

$\alpha y = 0.3$

ВЫХОД:

$\beta x = 8$ м

$\alpha x = 0$

$\beta y = 4$ м

$\alpha y = 0$

ФИЗИЧЕСКАЯ МОДЕЛЬ

Для моделирования используется линейная теория пучковой оптики.

Движение частицы описывается координатами:

x — поперечное смещение

x' — угол траектории

Эволюция координат при прохождении магнитного элемента описывается матрицей переноса.

Для учёта энергетического разброса используется расширенное описание с вектором:

$$(x, x', \delta),$$

где δ — относительное отклонение импульса.

Эволюция:

$$X' = M X$$

TWISS-ПАРАМЕТРЫ

TWISS ПАРАМЕТРЫ:

β — размер пучка

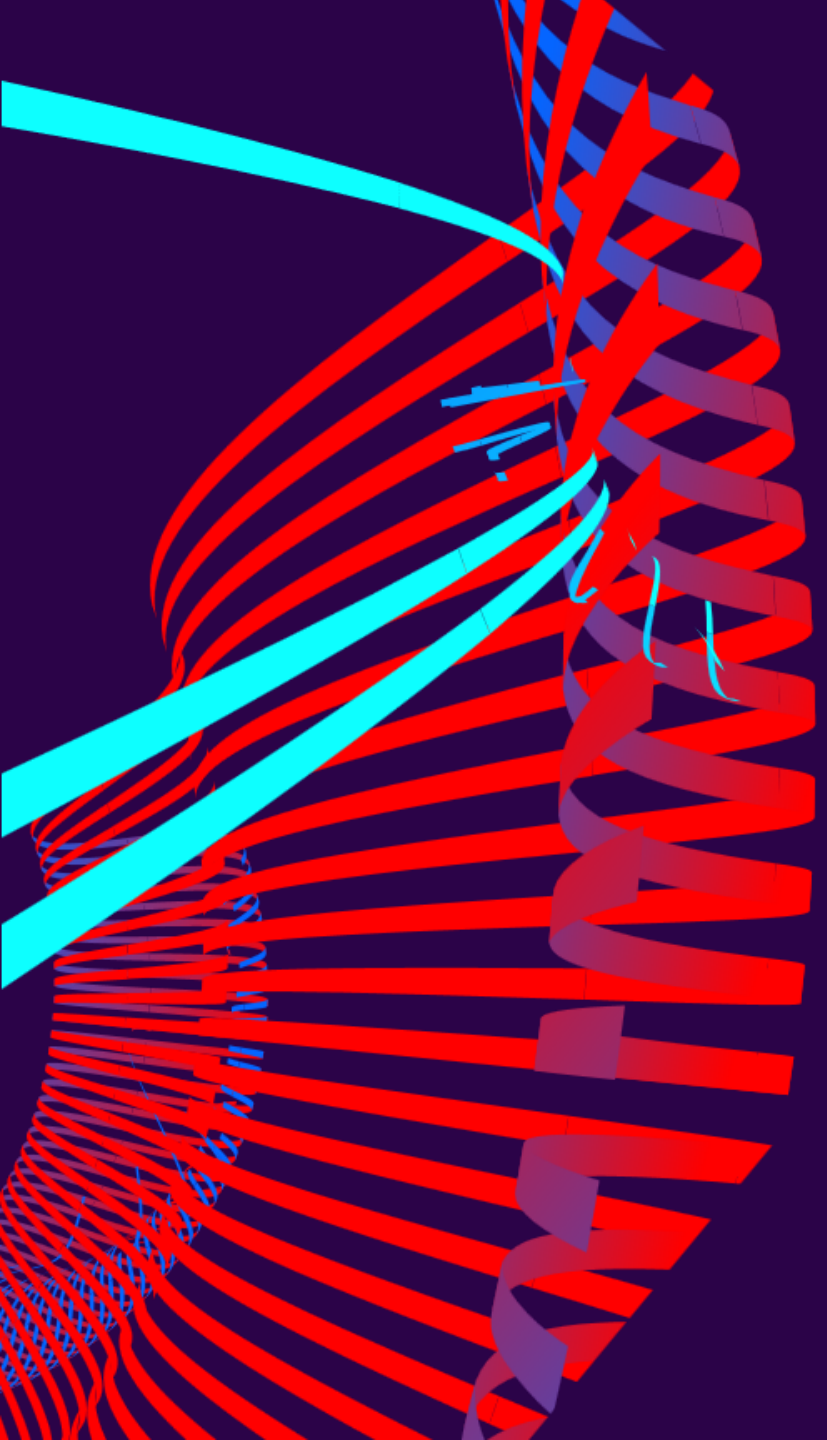
α — наклон фазового эллипса

TWISS-МАТРИЦА:

$T=\beta-\alpha-\alpha\,\gamma$

ЭВОЛЮЦИЯ:

$T' = M\,T\,M^T$



АРХИТЕКТУРА ПРОГРАММЫ

ОСНОВНЫЕ КОМПОНЕНТЫ

структуры данных пучка

генерация частиц

матрицы магнитных элементов

оптимизация параметров

трекинг пучка

визуализация

СТРУКТУРЫ ДАННЫХ

```
struct Particle {
    double x, xp;
    double y, yp;
    double dp;
};

struct BeamState {
    double s;
    double bx, ax, by, ay;
    double dx, dpx;
    int elem_type;
    vector<Particle> particles;
};
```

МАТРИЦЫ МАГНИТНЫХ ЭЛЕМЕНТОВ

```
Matx33d getDrift(double L) {
    return Matx33d(1.0, L, 0.0,
                   0.0, 1.0, 0.0,
                   0.0, 0.0, 1.0);
}
```

МАТРИЦА КВАДРУПОЛЯ

```
Matx33d getQuad(double L, double K, bool is_x) {
    if (std::abs(K) < 1e-9) return getDrift(L);
    double k_val = is_x ? K : -K;
    double r = std::sqrt(std::abs(k_val));
    if (k_val > 0) {
        return Matx33d(cos(r * L), (1.0 / r) * sin(r * L), 0.0,
                       -r * sin(r * L), cos(r * L), 0.0,
                       0.0, 0.0, 1.0);
    } else {
        return Matx33d(cosh(r * L), (1.0 / r) * sinh(r * L), 0.0,
                       r * sinh(r * L), cosh(r * L), 0.0,
                       0.0, 0.0, 1.0);
    }
}
```

Фокусировка:

cos, sin

Дефокусировка:

cosh, sinh

ГЕНЕРАЦИЯ ПУЧКА

```
vector<Particle> generateBeam(int N, double betaX, double alphaX, double betaY, double alphaY) {
    vector<Particle> beam;
    RNG rng;
    for (int i = 0; i < N; i++) {
        double u = rng.uniform(0.0, 1.0);
        double v = rng.uniform(0.0, 1.0);
        double r = sqrt(u);
        double theta = 2 * M_PI * v;

        double x = sqrt(eps_x * betaX) * r * cos(theta);
        double xp = -sqrt(eps_x / betaX) * (alphaX * r * cos(theta) + r * sin(theta));

        double y = sqrt(eps_y * betaY) * r * sin(theta);
        double yp = -sqrt(eps_y / betaY) * (alphaY * r * sin(theta) + r * cos(theta));

        double dp = rng.gaussian(1e-3);
        beam.push_back({x, xp, y, yp, dp});
    }
    return beam;
}
```

ОПТИМИЗАЦИЯ ПАРАМЕТРОВ

МЕТОД НЕЛДЕРА-МИДА

```
Ptr<DownhillSolver> solver = DownhillSolver::create();
solver->setFunction(makePtr<BeamMatchingObjective>());
Mat x = (Mat_<double>(1, 4) << 0.5, -0.5, 0.5, -0.5);
Mat step = (Mat_<double>(1, 4) << 0.05, 0.05, 0.05, 0.05);
solver->setInitStep(step);
solver->setTermCriteria(TermCriteria(TermCriteria::MAX_ITER + TermCriteria::EPS, 50000, 1e-16));

solver->minimize(x);
```

Оптимизируются параметры:
K_1, K_2, K_3, K_4 — градиенты квадрупольных магнитов.

ТРЕКИНГ

Основные функции:

```
track_drift(L);
track_quad(L, K);
```

Пучок проходит через последовательность элементов:
дрейф → квадруполь → дрейф → квадруполь → диполь.

РЕЗУЛЬТАТ ОПТИМИЗАЦИИ

```
Optimization Done.
K1 =  0.323813   ← слабо фокусирующая линза
K2 = -1.260350   ← сильно дефокусирующая
K3 =  1.314300   ← сильно фокусирующая
K4 = -0.889664   ← дефокусирующая

K5 (Dipole Angle) = 0.1 rad (фиксировано)

Final Dispersion Dx = 0.0504575 m
```

Результаты оптимизации:
Коэффициенты магнитов:
K_1
K_2
K_3
K_4

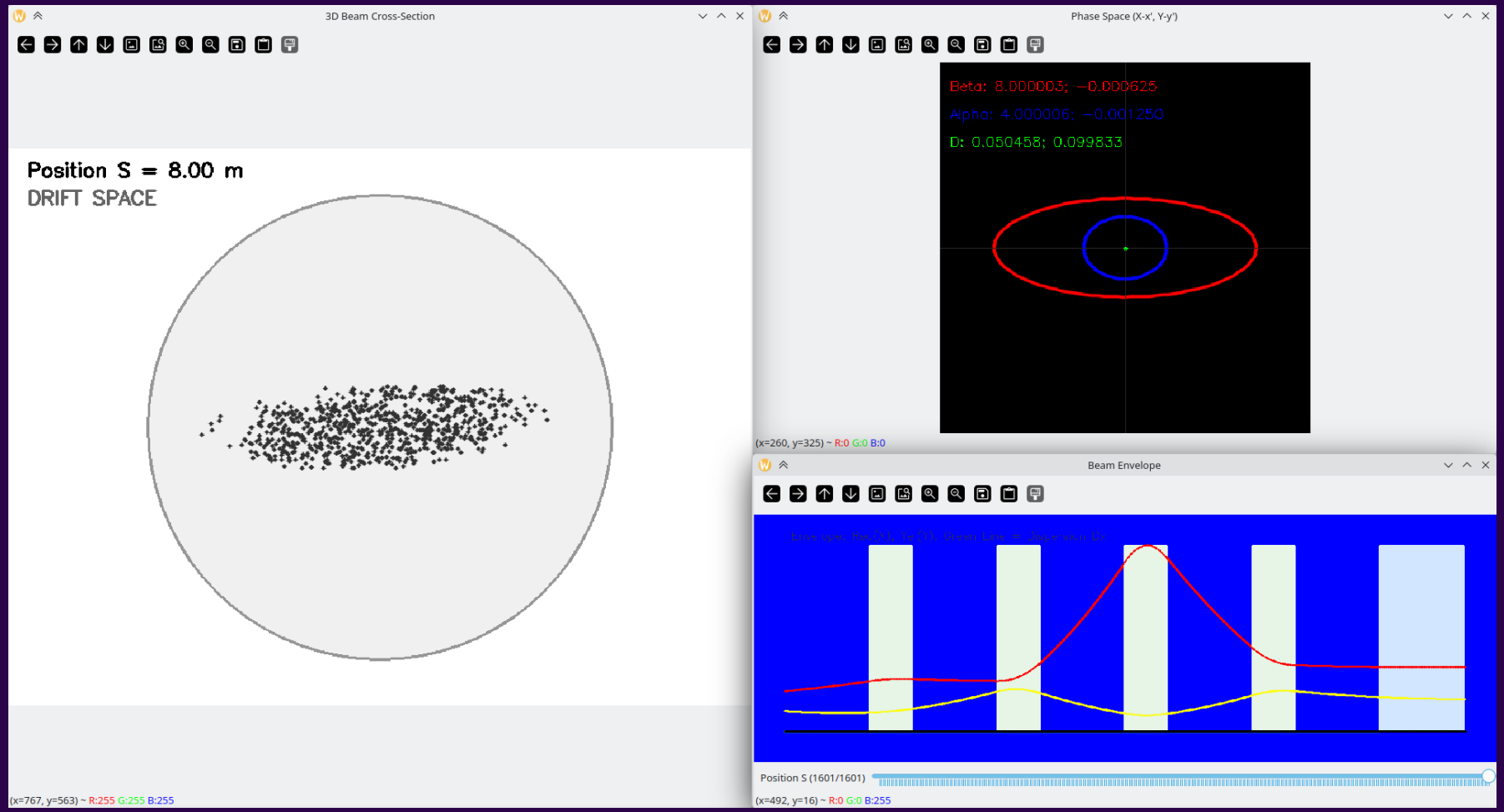
угол диполя: K5

Финальная дисперсия: Dx ≈ 0

ВИЗУАЛИЗАЦИЯ

3 окна программы:

- огибающая пучка
- фазовое пространство
- поперечное сечение



ЗАКЛЮЧЕНИЕ

Реализовано:

- модель линейной оптики
- генерация ансамбля частиц
- оптимизация параметров магнитов
- интерактивная визуализация

СПАСИБО